Marathwada Mitra Mandal's
COLLEGE OF ENGINEERING, Karvenagar, Pune - 52
An Autonomous Institute affiliated to SPPU
NAAC: A ++ | NBA Accredited **Best College Award 2019 SPPU**
website: www.mmcoe.edu.in email id: mmcoe@mmcoe.edu.in

A Project Report
On

Online Exam Processing Scheduler using FCFS, SJF and Round Robin

By

Mudra Chaughule-B24CE1133

Shubham Shinde-B24CE1116

Kaif Untwale-B25CE2013

Under the guidance of

Dr.Reshma Kapadi

Department of Computer Engineering

Marathwada Mitra Mandal's College of Engineering

Class: TY-II, Sem-I

Academic Year 2026-27

Date: 21/10/2026

CERTIFICATE

This is to certify that Mudra Chaughule-B24CE1133,Shubham Shinde-B24CE1116,Kaif Untwale-B25CE2013 of class TY Computer Engineering have successfully completed their project work on **"Online Exam Processing Scheduler using FCFS, SJF and Round Robin "** at MARATHWADA MITRA MANDAL'S COLLEGE OF ENGINEERING in the partial fulfillment of the Graduate Degree course in T.E at the Department of Computer Engineering in the academic Year 2026-2027 Semester –I.

Dr.Reshma Kapadi
Guide

Dr. S.S Shiravale
Head of the Department
Computer Engineering

Acknowledgement

I feel great pleasure in expressing my deepest sense of gratitude and sincere thanks to my guide **Dr.Reshma Kapadi** for their valuable guidance during the Project work, without which it would have been a very difficult task. I have no words to express my sincere thanks for valuable guidance, extreme assistance and cooperation extended to all the **Staff Members** of my Department.

This acknowledgement would be incomplete without expressing my special thanks to **Dr. S.S Shiravale**, Head of the Department (Computer Engineering) for their support during the work.

I would also like to extend my heartfelt gratitude to my **Principal, Dr. Sachin Sakhare** who provided a lot of valuable support, mostly being behind the veils of college bureaucracy.

Last but not least I would like to thank all the Teaching, Non- Teaching staff members of my Department, my parents and my colleagues who helped me directly or indirectly for completing this Project successfully.

Mudra Virendra Chaughule-B24CE1133

Shubham Shinde-B24CE1116

Kaif Untwale-B25CE2013

ABSTRACT

In an online examination, hundreds of students submit their answer sheets within a short window, and the evaluation server must decide the order in which these submissions are processed. This is exactly the CPU-scheduling problem studied in Operating Systems, where a submission plays the role of a process, its arrival time is the time it reaches the server, and its processing time is the CPU burst. A poor ordering makes some students wait unnecessarily long for their result.

This project implements an **Online Exam Processing Scheduler** in Java that simulates three classical scheduling algorithms: **First Come First Served (FCFS)**, **Shortest Job First (SJF, non-preemptive)** and **Round Robin (RR)** with a user-defined time quantum. The user enters the number of submissions together with their arrival and processing times and then chooses an algorithm from a menu. For each algorithm the program builds a Gantt chart (including CPU idle intervals), calculates completion, turnaround and waiting time of every submission, and prints the average waiting time and average turnaround time. Inputs are validated and invalid values are rejected.

The program was developed with core Java (ArrayList, Queue, Comparator and Scanner) and executed on the Windows command prompt. For three submissions with processing times 4, 3 and 2 arriving at time 0 and a quantum of 2, FCFS gave an average waiting time of 3.67, SJF gave 2.33 and Round Robin gave 4.67. SJF therefore produced the best average waiting time, FCFS was the simplest, and Round Robin gave the fairest sharing of the processor. The simulation helps in understanding and comparing the trade-offs of these algorithms.

MARATHWADA MITRAMANDAL, PUNE
'Welfare of Masses'
Est.1967

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Project Overview

1.2 Problem Definition

1.3 Motivation

1.4 Objectives

1.5 Scope of the Project

Chapter 2: Theoretical Background

2.1 Selected Concept

2.2 Relevant Terminology

2.3 Algorithm(s) / Technique(s)

2.4 Technologies and Tools Used

Chapter 3: System Analysis and Design

3.1 Functional Requirements

3.2 Hardware and Software Requirements

3.3 System Architecture / Block Diagram/Flowchart

3.5 Algorithm / Pseudocode

Chapter 4: Implementation

4.1 Implementation Approach

4.2 Module Description

4.3 Important Source Code with Explanation

4.4 Execution Procedure

4.5 Input and Output Screenshots

Chapter 5: Testing and Results

5.1 Test Cases

5.2 Actual Results

5.3 Result Analysis and Observations

5.4 Limitations

Conclusion

Future Scope

References

CHAPTER 1: INTRODUCTION

1.1 Project Overview

This mini project, **Online Exam Processing Scheduler**, is a console-based simulation written in Java that applies the **CPU scheduling** concept of Operating Systems to an online examination scenario. When students finish an online test they click "Submit", and the answer sheets (called *submissions* in this project) reach the evaluation server one after another. The server has limited processing capacity, so it must decide which submission is evaluated first. The scheduling algorithms of an operating system answer exactly this question.

The application accepts the number of submissions, the arrival time and the processing time of every submission and lets the user choose one of three scheduling algorithms: First Come First Served (FCFS), Shortest Job First (SJF – non-preemptive) and Round Robin (RR). For the chosen algorithm it prints a Gantt chart, a result table with waiting and turnaround time of each submission, and the average waiting and turnaround time, so that the algorithms can be compared on the same input.

1.2 Problem Definition

During an online exam, many submissions arrive at the evaluation server at different times and require different amounts of processing time. If they are processed in an unplanned order, short submissions may get stuck behind long ones, results are delayed, and the average waiting time of students increases. The problem solved by this project is: **given a set of submissions with arrival time and processing time, simulate FCFS, SJF and Round Robin scheduling and calculate the Gantt chart, waiting time and turnaround time so that the most suitable algorithm can be identified.**

This matters because scheduling directly affects the responsiveness and fairness of any shared system. Choosing the wrong policy leads to long queues, unhappy users and poor utilisation of the processor.

1.3 Motivation

CPU scheduling is a core topic of Systems Programming and Operating Systems, but the algorithms are often understood only through paper calculations. We wanted to convert this theory into a working program with a realistic, easily understood example. An online exam portal is something every student uses, so mapping

"submission = process" makes the concepts of arrival time, burst time, waiting time and turnaround time very intuitive. Building the simulator also gave us practice in Java collections, queue handling and structured program design.

1.4 Objectives

- To implement FCFS, non-preemptive SJF and Round Robin scheduling algorithms in Java.
- To accept arrival time and processing time for *n* submissions with proper input validation (n > 0, arrival ≥ 0, processing time > 0, quantum > 0).
- To generate a Gantt chart for each algorithm, including CPU idle intervals.
- To calculate completion time, turnaround time and waiting time of every submission, and the average waiting and turnaround time of each algorithm.
- To provide a menu-driven interface so that all three algorithms can be run repeatedly on the same data and their results compared.

1.5 Scope of the Project

Included: a menu-driven console simulator for three scheduling algorithms (FCFS, SJF non-preemptive, Round Robin with user-defined time quantum), arrival-time handling with idle-time detection, Gantt chart display, per-submission and average waiting/turnaround time.

Excluded: preemptive SJF (SRTF), priority scheduling, multi-level queues, context-switch overhead, I/O bursts, multiple processors and a graphical user interface. All times are whole numbers of abstract time units.

Intended use: as a learning and demonstration tool for the SPOS/Operating Systems course, and as a base for comparing scheduling policies for an online examination server.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Selected SPOS Concept

The selected concept is **CPU (process) scheduling**. In a multiprogramming operating system many processes are ready to run, but the CPU can execute only one at a time. The **short-term scheduler** selects one process from the ready queue and the dispatcher gives it the CPU. The policy used for this selection is the *scheduling algorithm*, and it decides how long processes wait, how quickly they finish and how fairly the CPU is shared.

Scheduling algorithms are classified as **non-preemptive** (once a process gets the CPU it keeps it until it finishes, e.g., FCFS and the SJF version used here) and **preemptive** (the running process can be interrupted, e.g., Round Robin when its time quantum expires). In this project each online-exam submission is treated as a process that needs a certain amount of processing time on the evaluation server.

2.2 Relevant Terminology

Term	Meaning in this project
Submission / Process	An answer sheet (process) waiting to be evaluated by the server; represented by the Submission class (id, arrival, burst, remaining, waiting, turnaround, completion).
Arrival Time (AT)	The time at which a submission reaches the server and enters the ready queue.
Burst / Processing Time (BT)	Total processing time required by the submission.
Completion Time (CT)	The time at which the submission finishes processing.
Turnaround Time (TAT)	Total time spent in the system: TAT = CT − AT.
Waiting Time (WT)	Time spent waiting in the ready queue: WT = TAT − BT.
Time Quantum	Maximum continuous time a submission may run in Round Robin before it is preempted.
Ready Queue	FIFO queue (Queue<Integer> / LinkedList) holding submissions that have arrived and are waiting for the CPU.
Gantt Chart	Timeline showing which submission ran during each time interval, including IDLE periods.
IDLE	Interval during which no submission has arrived and the CPU has nothing to run.
Convoy Effect	Short jobs waiting behind a long job, a drawback of FCFS.
Starvation	Indefinite postponement of a long job when shorter jobs keep arriving (possible in SJF).

2.3 Algorithm(s) / Technique(s)

(a) First Come First Served (FCFS)

FCFS is the simplest, non-preemptive algorithm. Submissions are sorted by arrival time and processed strictly in that order. If the CPU becomes free before the next submission has arrived, an IDLE slot is recorded. *Assumption:* submissions with equal arrival time are served in their entry order (stable sort). *Steps:* sort by arrival; for each submission start = max(current time, arrival); WT = start − arrival; CT = start + burst; TAT = CT − AT. Its drawback is the convoy effect.

(b) Shortest Job First (non-preemptive)

Whenever the CPU becomes free, SJF chooses, among the submissions that have already arrived, the one with the **smallest processing time**; it then runs to completion. If no submission has arrived, the CPU stays idle for one time unit and the check is repeated. SJF gives the minimum average waiting time for a given set of jobs but needs the burst time in advance and may starve long jobs. *Assumption:* ties are broken in favour of the submission that was entered first.

(c) Round Robin (RR)

Round Robin is a preemptive algorithm designed for time-sharing. A ready queue is maintained and each submission is given the CPU for at most one **time quantum**. If it is not finished, it is placed at the end of the queue; otherwise its completion time is recorded. New arrivals that come during a time slice are added to the queue before the preempted submission is re-queued. Waiting time is computed as WT = TAT − BT. RR avoids starvation and gives good response time, but a very small quantum increases overhead and a very large quantum makes it behave like FCFS.

2.4 Technologies and Tools Used

Tool / Technology	Purpose
Java (JDK)	Programming language used to implement the simulator (object-oriented, collections framework).
javac / java	Compiler and Java Virtual Machine used to compile and run the program.
java.util package	ArrayList, Queue, LinkedList, Comparator and Scanner for data storage, ready queue, sorting and input.
Windows 10/11 Command Prompt	Terminal in which the program is executed and the output captured.
Text editor / IDE	Used to write and edit the source code (any Java editor, e.g., VS Code, Notepad++ or IntelliJ).

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Functional Requirements

The program must:

1. Accept the number of submissions *n* and reject values less than or equal to zero.
2. Accept the arrival time and processing time of each submission and reject negative arrival time or non-positive processing time.
3. Display a menu with the options 1. FCFS, 2. SJF, 3. Round Robin and 4. Exit, and keep running until the user exits.
4. Run the selected algorithm on a copy of the input data so that the original data is preserved for the next run.
5. For Round Robin, accept a time quantum and reject a non-positive value.
6. Generate and display the Gantt chart with start and end times, including IDLE intervals.
7. Calculate and display waiting time and turnaround time of every submission and their averages (two decimal places).
8. Report invalid menu choices without crashing.

3.2 Hardware and Software Requirements

Hardware: any computer with a processor of 1 GHz or higher, minimum 2 GB RAM (4 GB recommended) and about 200 MB free storage for the JDK and source files.

Software: Windows / Linux / macOS operating system; Java Development Kit (JDK 8 or higher); command prompt or terminal; any text editor or IDE (VS Code, Notepad++, IntelliJ IDEA, Eclipse).

3.3 System Architecture / Block Diagram/FlowChart

The system follows a simple pipeline. User input is validated and stored as Submission objects. The menu controller passes a copy of this data to the selected scheduling module (FCFS, SJF or Round Robin). Every module uses the common Gantt-chart and time-calculation routines to produce the output.

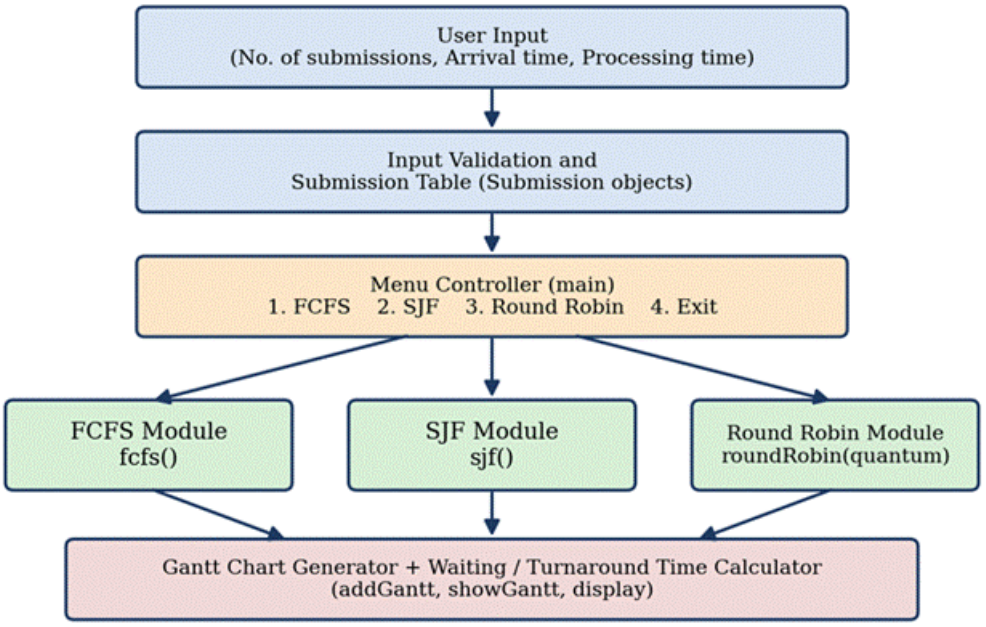


Figure 3.1: Block diagram of the Online Exam Processing Scheduler

The flowchart below shows the overall control flow of the program, from input and validation to the menu loop and the individual scheduling modules.

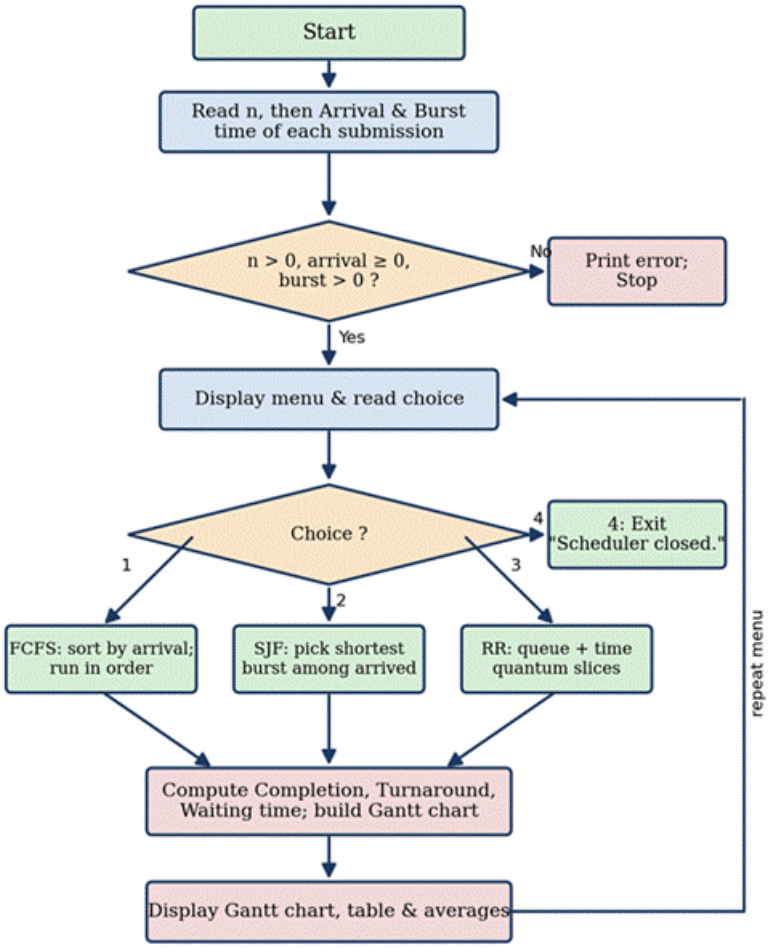


Figure 3.2: Flowchart of the program

3.5 Algorithm / Pseudocode

Main program

1. Read n. If n ≤ 0, print "Invalid number of submissions." and stop.
2. For i = 1 to n: read arrival[i] and burst[i]; if arrival < 0 or burst ≤ 0, print "Invalid input." and stop; otherwise create Submission Si.
3. Repeat: display menu and read choice.
4. If choice = 1 call FCFS; if 2 call SJF; if 3 read quantum (must be > 0) and call RoundRobin; if 4 print "Scheduler closed." and exit; otherwise print "Invalid choice."

FCFS

```
copy submissions; sort by arrival time
time = 0
for each submission s in sorted order:
         if time < s.arrival:
         record IDLE from time to s.arrival; time = s.arrival
         begin = time
         s.waiting    = time - s.arrival
         time         = time + s.burst
         s.completion = time
         s.turnaround = time - s.arrival
         record s in Gantt chart (begin, time)
print Gantt chart, table and averages
```

SJF (non-preemptive)

```
time = 0; completed = 0
while completed < n:
          pick = the not-done submission with arrival <= time and smallest burst
          if no such submission:
          record IDLE (time, time+1); time = time + 1; continue
          begin = time
          pick.waiting           = time - pick.arrival
          time                   = time + pick.burst
          pick.completion = time
          pick.turnaround = time - pick.arrival
          mark pick done; completed = completed + 1; record in Gantt chart
print Gantt chart, table and averages
```

Round Robin

```
time = 0; completed = 0; queue = empty
while completed < n:
          add every not-yet-added submission with arrival <= time to queue
          if queue is empty:
          record IDLE (time, time+1); time = time + 1; continue
          s = queue.poll()
          run = min(quantum, s.remaining)
          time = time + run; s.remaining = s.remaining - run
          record s in Gantt chart (time-run, time)
          add every not-yet-added submission with arrival <= time to queue
          if s.remaining > 0:  queue.add(s)
          else:
          s.completion = time
          s.turnaround = time - s.arrival
          s.waiting   = s.turnaround - s.burst
          completed = completed + 1
print Gantt chart, table and averages
```

CHAPTER 4: IMPLEMENTATION

4.1 Implementation Approach

The solution is implemented in a single Java class, **OnlineExamScheduler**. Each online-exam submission is represented by a nested static class **Submission** that stores its id, arrival time, burst (processing) time, remaining time and the calculated completion, turnaround and waiting times. All submissions are kept in an `ArrayList<Submission>`.

Every scheduling algorithm is a separate static method that first makes a copy of the input list (`copy()`), so that results of one algorithm never affect another. FCFS uses sorting with a `Comparator`; SJF uses a linear search over arrived, unfinished submissions; Round Robin uses a `Queue<Integer>` (LinkedList) as the ready queue and a `remaining` field that is reduced by one quantum at a time. The Gantt chart is stored in three parallel lists (names, start times, end times), and consecutive IDLE slots are merged into one. A `while(true)` loop with a `switch` statement provides the menu.

4.2 Module Description

Module / Function	Responsibility
Submission (class)	Data holder for one submission: id, arrival, burst, remaining, waiting, turnaround, completion.
main()	Reads and validates input, creates the Submission list, runs the menu loop and calls the selected algorithm.
copy()	Creates a fresh copy of the submission list (resetting remaining time) for each algorithm run.
fcfs()	Implements First Come First Served scheduling with idle-time handling.

sjf()	Implements non-preemptive Shortest Job First scheduling with idle-time handling.
roundRobin()	Implements Round Robin scheduling using a ready queue and a user-defined time quantum.
addGantt()	Adds an entry (name, start, end) to the Gantt chart and merges consecutive IDLE entries.
showGantt()	Prints the Gantt chart with the time markers below it.
display()	Prints the result table (ID, arrival, burst, waiting, turnaround) and the average waiting and turnaround time.

4.3 Important Source Code with Explanation

Key parts of the program are shown below. The complete source code is given in the Appendix.

(i) Submission class

```
static class Submission {
        int id, arrival, burst, remaining;
        int waiting, turnaround, completion;

        Submission(int id, int arrival, int burst) {
        this.id = id;
        this.arrival = arrival;
        this.burst = burst;
        remaining = burst;
        }
}
```

Each submission stores its own timing data. The field *remaining* is initialised with the burst time and is used only by Round Robin.

(ii) Gantt chart helper

```
static void addGantt(ArrayList<String> chart, ArrayList<Integer> start,
                     ArrayList<Integer> end, String name, int a, int b) {
        if (name.equals("IDLE") && !chart.isEmpty()
        && chart.get(chart.size() - 1).equals("IDLE")) {
        end.set(end.size() - 1, b);                  // merge consecutive IDLE slots
        } else {
        chart.add(name);
        start.add(a);
        end.add(b);
        }
}
```

The chart is kept as three parallel lists. When two IDLE slots follow each other (because SJF and RR advance one time unit at a time while idle) they are merged so that the chart shows a single IDLE block.

(iii) FCFS scheduling

```
static void fcfs(ArrayList<Submission> data) {
        ArrayList<Submission> list = copy(data);
        list.sort(Comparator.comparingInt(s -> s.arrival));
        ...
        int time = 0;
        for (Submission s : list) {
        if (time < s.arrival) {                      // CPU idle until next arrival
        addGantt(chart, start, end, "IDLE", time, s.arrival);
        time = s.arrival;
        }
        int begin = time;
        s.waiting = time - s.arrival;
        time += s.burst;
        s.completion = time;
        s.turnaround = time - s.arrival;
        addGantt(chart, start, end, "S" + s.id, begin, time);
        }
        showGantt(chart, start, end);
        display("FCFS Result", list);
}
```

The submissions are sorted by arrival time. If the clock is behind the next arrival, an IDLE interval is recorded and the clock jumps to the arrival time. Waiting time is the time between arrival and start; turnaround time is completion minus arrival.

(iv) SJF scheduling (non-preemptive)

```
while (completed < n) {
          int index = -1;
          for (int i = 0; i < n; i++) {
          Submission s = list.get(i);
          if (!done[i] && s.arrival <= time &&
          (index == -1 || s.burst < list.get(index).burst)) {
          index = i;                          // shortest arrived job so far
          }
          }
          if (index == -1) {                            // nothing has arrived yet
          addGantt(chart, start, end, "IDLE", time, time + 1);
          time++;
          continue;
          }
          Submission s = list.get(index);
          int begin = time;
          s.waiting = time - s.arrival;
          time += s.burst;                              // non-preemptive: run to end
          s.completion = time;
          s.turnaround = time - s.arrival;
          addGantt(chart, start, end, "S" + s.id, begin, time);
          done[index] = true;
          completed++;
}
```

At every decision point the loop selects the unfinished submission that has already arrived and has the smallest burst time (strict "<" keeps the earlier-entered submission on a tie). If none has arrived, one IDLE unit is recorded. The selected submission runs to completion.

(v) Round Robin scheduling

```
int index = q.poll();                           // take head of ready queue
Submission s = list.get(index);
int begin = time;
int run = Math.min(quantum, s.remaining);       // run for one quantum at most
time += run;
s.remaining -= run;
addGantt(chart, start, end, "S" + s.id, begin, time);

for (int i = 0; i < n; i++) {                   // admit new arrivals first
          if (!added[i] && list.get(i).arrival <= time) {
          q.add(i);
          added[i] = true;
          }
}
if (s.remaining > 0) {
          q.add(index);                                 // not finished: back of queue
} else {
          s.completion = time;
          s.turnaround = time - s.arrival;
          s.waiting = s.turnaround - s.burst;
          completed++;
}
```

The head of the queue runs for min(quantum, remaining). New arrivals during the slice are queued first, and then the preempted submission is placed at the tail if it still has remaining time. When it finishes, turnaround time is completion minus arrival and waiting time is turnaround minus burst.

(vi) Input handling and menu

```
System.out.print("Enter number of submissions: ");
int n = sc.nextInt();
if (n <= 0) { System.out.println("Invalid number of submissions."); return; }
for (int i = 1; i <= n; i++) {
          System.out.print("Arrival time: ");           int arrival = sc.nextInt();
          System.out.print("Processing time: ");  int burst   = sc.nextInt();
          if (arrival < 0 || burst <= 0) { System.out.println("Invalid input."); return; }
          list.add(new Submission(i, arrival, burst));
}
while (true) {                                  // menu loop
          ...
          switch (choice) {
          case 1: fcfs(list); break;
          case 2: sjf(list);  break;
          case 3: /* read quantum (> 0) */ roundRobin(list, quantum); break;
          case 4: System.out.println("Scheduler closed."); sc.close(); return;
          default: System.out.println("Invalid choice.");
```

```
        }
}
```

The input is validated immediately; invalid values terminate the program with a message. The menu loop runs until option 4 is selected, and the same input data can be used for all three algorithms.

4.4 Execution Procedure

1. Install JDK 8 or higher and verify with `java -version`.
2. Save the program as **OnlineExamScheduler.java** (the file name must match the public class name).
3. Open Command Prompt / terminal in the folder containing the file.
4. Compile: `javac OnlineExamScheduler.java`
5. Run: `java OnlineExamScheduler`
6. Enter the number of submissions, then the arrival time and processing time of each submission.
7. Choose 1 (FCFS), 2 (SJF) or 3 (Round Robin; also enter the time quantum). Repeat for other algorithms and choose 4 to exit.

4.5 Input and Output Screenshots

The program was run with three submissions: S1 (arrival 0, processing time 4), S2 (arrival 0, processing time 3) and S3 (arrival 0, processing time 2). FCFS, SJF and Round Robin (quantum = 2) were executed one after another.

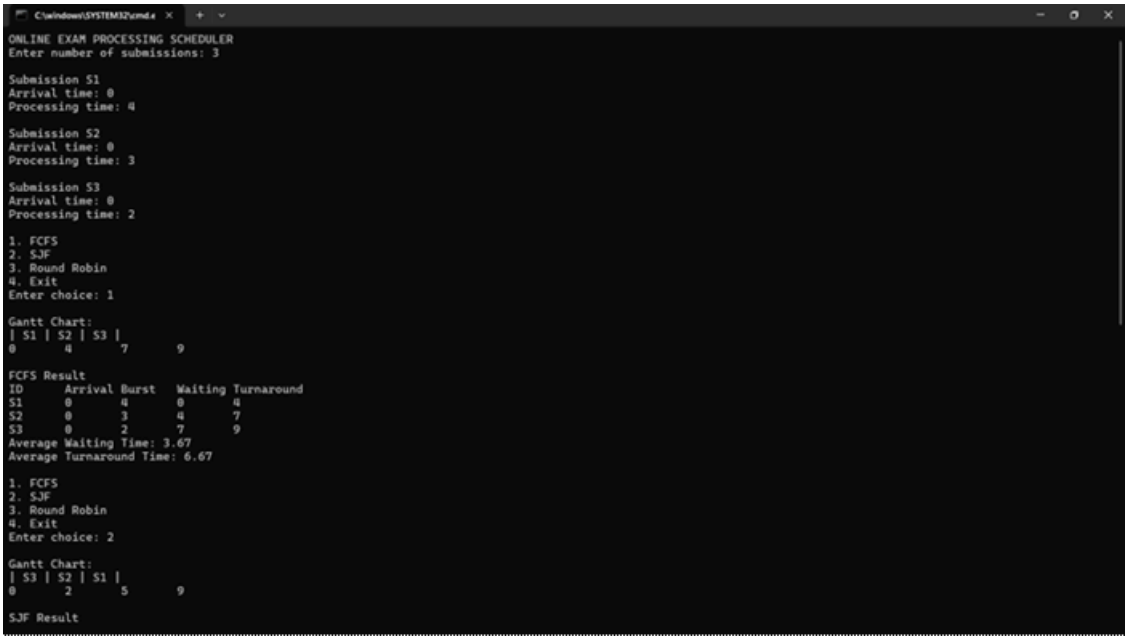


Figure 4.1: Input of submissions, FCFS output and SJF Gantt chart

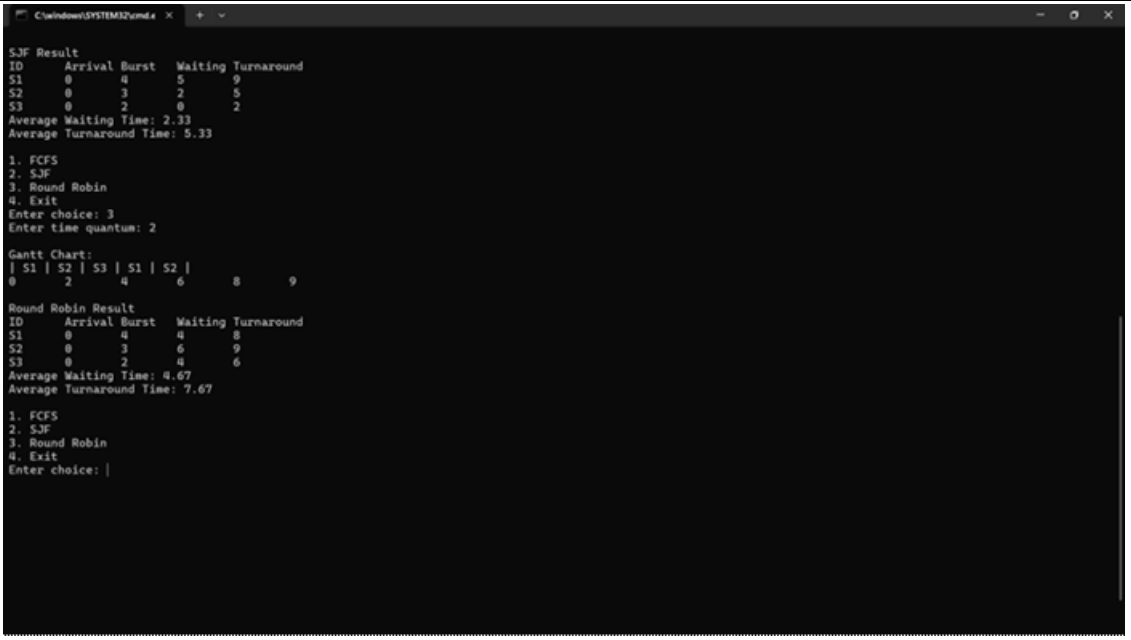


Figure 4.2: SJF result table and Round Robin (quantum = 2) output

CHAPTER 5: TESTING AND RESULTS

5.1 Test Cases

Test Case ID	Input / Scenario	Expected Output	Actual Output	Status
TC1	n = 3; S1(0,4), S2(0,3), S3(0,2); choice 1 (FCFS)	Gantt: S1 S2 S3 (0, 4, 7, 9); Avg WT = 3.67; Avg TAT = 6.67	Gantt: | S1 | S2 | S3 | 0 4 7 9; Avg WT = 3.67; Avg TAT = 6.67	Pass
TC2	Same data; choice 2 (SJF)	Gantt: S3 S2 S1 (0, 2, 5, 9); Avg WT = 2.33; Avg TAT = 5.33	Gantt: | S3 | S2 | S1 | 0 2 5 9; Avg WT = 2.33; Avg TAT = 5.33	Pass
TC3	Same data; choice 3 (RR), quantum = 2	Gantt: S1 S2 S3 S1 S2 (0, 2, 4, 6, 8, 9); Avg WT = 4.67; Avg TAT = 7.67	Gantt: | S1 | S2 | S3 | S1 | S2 | 0 2 4 6 8 9; Avg WT = 4.67; Avg TAT = 7.67	Pass
TC4	Boundary: n = 1; S1(0,5); FCFS	Gantt: S1 (0, 5); WT = 0; TAT = 5	Gantt: | S1 | 0 5; WT = 0; TAT = 5	Pass
TC5	Idle time: n = 2; S1(0,2), S2(5,3); FCFS	Gantt: S1 IDLE S2 (0, 2, 5, 8); Avg WT = 0.00; Avg TAT = 2.50	Gantt: | S1 | IDLE | S2 | 0 2 5 8; Avg WT = 0.00; Avg TAT = 2.50	Pass
TC6	Invalid: n = 0	Error message and program stops	"Invalid number of submissions."	Pass
TC7	Invalid: arrival = −1 or processing time = 0	Error message and program stops	"Invalid input."	Pass
TC8	Invalid: Round Robin with quantum = 0	Error message; menu is shown again	"Quantum must be positive."	Pass
TC9	Invalid menu choice (e.g., 7)	Error message; menu is shown again	"Invalid choice."	Pass

5.2 Actual Results

Normal input (S1, S2, S3 with processing times 4, 3, 2, all arriving at time 0):

Algorithm	Gantt Chart	Avg Waiting Time	Avg Turnaround Time	Order	Quantum
FCFS	0-4 S1, 4-7 S2, 7-9 S3	3.67	6.67	S1, S2, S3	–
SJF	0-2 S3, 2-5 S2, 5-9 S1	2.33	5.33	S3, S2, S1	–
Round Robin	0-2 S1, 2-4 S2, 4-6 S3, 6-8 S1, 8-9 S2	4.67	7.67	S1, S2, S3, S1, S2	2

Per-submission results (from the program output):

Submission	Arrival	Burst	FCFS WT / TAT	SJF WT / TAT	RR WT / TAT
S1	0	4	0 / 4	5 / 9	4 / 8
S2	0	3	4 / 7	2 / 5	6 / 9
S3	0	2	7 / 9	0 / 2	4 / 6

Boundary and invalid inputs: a single submission (n = 1) is scheduled without waiting; a gap between arrivals is shown as an IDLE block in the Gantt chart; n ≤ 0, negative arrival time, zero processing time, zero quantum and invalid menu choices are rejected with an appropriate message (see test cases TC4–TC9).

5.3 Result Analysis and Observations

Manual verification. With all submissions arriving at time 0, waiting time equals the start time and turnaround time equals the completion time.

- **FCFS:** S1 runs 0–4, S2 runs 4–7, S3 runs 7–9. WT = 0, 4, 7 → average = 11/3 = **3.67**; TAT = 4, 7, 9 → average = 20/3 = **6.67**.
- **SJF:** order of burst times 2, 3, 4, i.e., S3 (0–2), S2 (2–5), S1 (5–9). WT = 0, 2, 5 → average = 7/3 = **2.33**; TAT = 2, 5, 9 → average = 16/3 = **5.33**.
- **Round Robin (q = 2):** S1 (0–2), S2 (2–4), S3 (4–6, finishes), S1 (6–8, finishes), S2 (8–9, finishes). TAT = 8, 9, 6 → average = 23/3 = **7.67**; WT = TAT − BT = 4, 6, 4 → average = 14/3 = **4.67**.

All values calculated by hand match the program output shown in Figures 4.1 and 4.2.

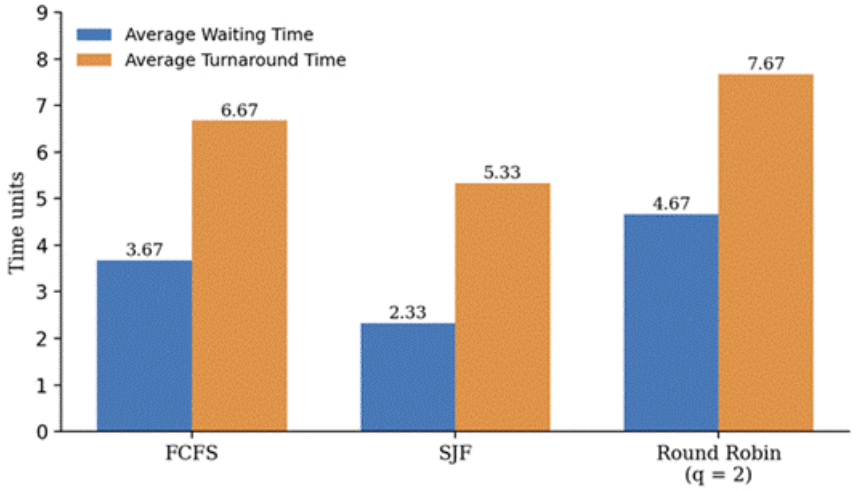


Figure 5.1: Comparison of average waiting and turnaround time

Observations:

- SJF gave the lowest average waiting and turnaround time (2.33 and 5.33), confirming that it is optimal for average waiting time when all jobs are available together.
- FCFS was in between (3.67 and 6.67); the longest submission S1 delayed S2 and S3, a small example of the convoy effect.
- Round Robin had the highest averages (4.67 and 7.67) for this input because the submissions are short and the quantum causes them to be interleaved. Its advantage is fairness: no submission waits for another to finish completely, and no starvation can occur.
- For an online exam server, SJF is suitable when processing times can be estimated in advance, while Round Robin is preferable where fairness and response time are more important than the average waiting time.

5.4 Limitations

- SJF is non-preemptive only; the preemptive variant (SRTF) and priority scheduling are not implemented.
- SJF assumes that the processing time of every submission is known in advance, and it can starve long submissions.
- Context-switch time, I/O waiting and multiple processors are not modelled; all times are whole-number time units.
- The interface is console based; non-numeric input (e.g., letters) is not handled and invalid values terminate the program instead of asking again.
- Results are shown only for the console; no export to file or graphical Gantt chart is available.

CONCLUSION

The Online Exam Processing Scheduler was successfully designed and implemented in Java. It simulates FCFS, non-preemptive SJF and Round Robin scheduling for online-exam submissions, validates user input, displays Gantt charts including idle time, and calculates waiting and turnaround times along with their averages. The output was verified against manual calculations, and all objectives of the project were achieved.

The experiment showed that SJF gives the lowest average waiting time (2.33), FCFS is simple but suffers from the convoy effect (3.67), and Round Robin shares the processor fairly but increased the averages for the given input (4.67). Through this project we gained practical understanding of CPU scheduling, ready queues, Gantt charts and performance metrics, and of structuring a menu-driven Java program using collections.

FUTURE SCOPE

- Add preemptive SJF (SRTF), priority scheduling and multilevel queue scheduling for further comparison.
- Add a graphical user interface with a visual Gantt chart and charts of average waiting/turnaround time.
- Read submissions from a file or database and export the results to CSV/PDF.
- Include context-switch overhead, response time, throughput and CPU utilisation metrics.
- Re-prompt on invalid input and handle non-numeric input with exception handling.
- Extend the simulation to multiple evaluation servers (multiprocessor scheduling).

REFERENCES

[1] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.

[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed., Pearson, 2018.

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2015.

[4] Oracle, “Java Platform, Standard Edition Documentation – java.util package (ArrayList, Queue, LinkedList, Comparator, Scanner).” [Online]. Available: https://docs.oracle.com/en/java/javase/

[5] Oracle, “The Java Tutorials.” [Online]. Available: https://docs.oracle.com/javase/tutorial/

APPENDIX: COMPLETE SOURCE CODE

File name: OnlineExamScheduler.java

```
import java.util.*;
public class OnlineExamScheduler {
         static class Submission {
         int id, arrival, burst, remaining;
         int waiting, turnaround, completion;
         Submission(int id, int arrival, int burst) {
         this.id = id;
         this.arrival = arrival;
         this.burst = burst;
         remaining = burst;
         }
         }
         static Scanner sc = new Scanner(System.in);
         // Store and display Gantt Chart
         static void addGantt(ArrayList<String> chart,
                        ArrayList<Integer> start,
                        ArrayList<Integer> end,
                               String name, int a, int b) {
         if (name.equals("IDLE") && !chart.isEmpty()
                    && chart.get(chart.size() - 1).equals("IDLE")) {
         end.set(end.size() - 1, b);
         } else {
         chart.add(name);
         start.add(a);
         end.add(b);
         }
         }
         static void showGantt(ArrayList<String> chart,
                         ArrayList<Integer> start,
                         ArrayList<Integer> end) {
         System.out.println("\nGantt Chart:");
         for (int i = 0; i < chart.size(); i++) {
         System.out.print("| " + chart.get(i) + " ");
         }
         System.out.println("|");
         if (!chart.isEmpty()) {
         System.out.print(start.get(0));
         for (int i = 0; i < end.size(); i++) {
                    System.out.print("\t" + end.get(i));
         }
         System.out.println();
         }
         }
         static void display(String name, ArrayList<Submission> list) {
         double wt = 0, tat = 0;
         System.out.println("\n" + name);
        System.out.println("ID\tArrival\tBurst\tWaiting\tTurnaround");
         for (Submission s : list) {
         System.out.println("S" + s.id + "\t" + s.arrival
                    + "\t" + s.burst + "\t" + s.waiting
                    + "\t" + s.turnaround);
         wt += s.waiting;
         tat += s.turnaround;
         }
         System.out.printf("Average Waiting Time: %.2f\n",
                    wt / list.size());
         System.out.printf("Average Turnaround Time: %.2f\n",
                    tat / list.size());
         }
         static ArrayList<Submission> copy(ArrayList<Submission> data) {
         ArrayList<Submission> list = new ArrayList<>();
         for (Submission s : data) {
         list.add(new Submission(s.id, s.arrival, s.burst));
         }
         return list;
         }
         // FCFS Scheduling
```

```
static void fcfs(ArrayList<Submission> data) {
ArrayList<Submission> list = copy(data);
list.sort(Comparator.comparingInt(s -> s.arrival));
ArrayList<String> chart = new ArrayList<>();
ArrayList<Integer> start = new ArrayList<>();
ArrayList<Integer> end = new ArrayList<>();
int time = 0;
for (Submission s : list) {
if (time < s.arrival) {
            addGantt(chart, start, end, "IDLE",
            time, s.arrival);
            time = s.arrival;
}
int begin = time;
s.waiting = time - s.arrival;
time += s.burst;
s.completion = time;
s.turnaround = time - s.arrival;
addGantt(chart, start, end, "S" + s.id, begin, time);
}
showGantt(chart, start, end);
display("FCFS Result", list);
}
// SJF Non-preemptive Scheduling
static void sjf(ArrayList<Submission> data) {
ArrayList<Submission> list = copy(data);
int n = list.size(), time = 0, completed = 0;
boolean[] done = new boolean[n];
ArrayList<String> chart = new ArrayList<>();
ArrayList<Integer> start = new ArrayList<>();
ArrayList<Integer> end = new ArrayList<>();
while (completed < n) {
int index = -1;
for (int i = 0; i < n; i++) {
            Submission s = list.get(i);
            if (!done[i] && s.arrival <= time &&
            (index == -1 ||
            s.burst < list.get(index).burst)) {
            index = i;
            }
}
if (index == -1) {
            addGantt(chart, start, end, "IDLE",
            time, time + 1);
            time++;
            continue;
}
Submission s = list.get(index);
int begin = time;
s.waiting = time - s.arrival;
time += s.burst;
s.completion = time;
s.turnaround = time - s.arrival;
addGantt(chart, start, end, "S" + s.id, begin, time);
done[index] = true;
completed++;
}
showGantt(chart, start, end);
display("SJF Result", list);
}
// Round Robin Scheduling
static void roundRobin(ArrayList<Submission> data, int quantum) {
ArrayList<Submission> list = copy(data);
Queue<Integer> q = new LinkedList<>();
boolean[] added = new boolean[list.size()];
ArrayList<String> chart = new ArrayList<>();
ArrayList<Integer> start = new ArrayList<>();
ArrayList<Integer> end = new ArrayList<>();
int time = 0, completed = 0, n = list.size();
while (completed < n) {
for (int i = 0; i < n; i++) {
            if (!added[i] && list.get(i).arrival <= time) {
            q.add(i);
```

```
                added[i] = true;
                }
        }
        if (q.isEmpty()) {
                addGantt(chart, start, end, "IDLE",
                time, time + 1);
                time++;
                continue;
        }
        int index = q.poll();
        Submission s = list.get(index);
        int begin = time;
        int run = Math.min(quantum, s.remaining);
        time += run;
        s.remaining -= run;
        addGantt(chart, start, end, "S" + s.id, begin, time);
        for (int i = 0; i < n; i++) {
                if (!added[i] && list.get(i).arrival <= time) {
                q.add(i);
                added[i] = true;
                }
        }
        if (s.remaining > 0) {
                q.add(index);
        } else {
                s.completion = time;
                s.turnaround = time - s.arrival;
                s.waiting = s.turnaround - s.burst;
                completed++;
        }
        }
        showGantt(chart, start, end);
        display("Round Robin Result", list);
        }
        public static void main(String[] args) {
        ArrayList<Submission> list = new ArrayList<>();
        System.out.println("ONLINE EXAM PROCESSING SCHEDULER");
        System.out.print("Enter number of submissions: ");
        int n = sc.nextInt();
        if (n <= 0) {
        System.out.println("Invalid number of submissions.");
        return;
        }
        for (int i = 1; i <= n; i++) {
          System.out.println("\nSubmission S" + i);
        System.out.print("Arrival time: ");
        int arrival = sc.nextInt();
        System.out.print("Processing time: ");
        int burst = sc.nextInt();
        if (arrival < 0 || burst <= 0) {
              System.out.println("Invalid input.");
                return;
        }
        list.add(new Submission(i, arrival, burst));
        }
        while (true) {
        System.out.println("\n1. FCFS");
        System.out.println("2. SJF");
        System.out.println("3. Round Robin");
        System.out.println("4. Exit");
        System.out.print("Enter choice: ");
        int choice = sc.nextInt();
        switch (choice) {
                case 1:
                fcfs(list);
                break;
                case 2:
                sjf(list);
                break;
                case 3:
               System.out.print("Enter time quantum: ");
                int quantum = sc.nextInt();
                if (quantum > 0)
```

```
                    roundRobin(list, quantum);
                    else
                       System.out.println("Quantum must be positive.");
                    break;
                    case 4:
                   System.out.println("Scheduler closed.");
                    sc.close();
                    return;
                    default:
                   System.out.println("Invalid choice.");
            }
            }
            }
}
```

